Give feedback
Ask for help
Useful links

# Introduction to pandas

## https://bioinfmsc8.bio.ed.ac.uk/AY25_BPSM17.html

---

## Quick links:
## Click on any ⌂ to get back here!



As always, there are many ways to do things in Python (and other languages), but `pandas` is almost always our first choice for dealing with tabular data objects in Python. If we have to process any data that are like Excel tables, pandas is probably **the** way.

It has its own web site: https://pandas.pydata.org/

The name `pandas` has little to do with the cute black and white bears!
Two years ago, we could go see Yang Guang at the Edinburgh Zoo, either by visiting the Zoo or watching the live panda webcam!

Give feedback
Ask for help

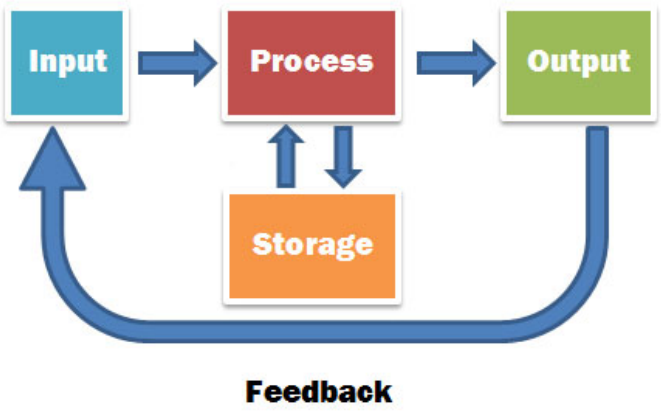
Last year, the attraction was Haggis, the pygmy hippo baby/calf, now moved to a different place!

This year, starting yesterday, the attraction is a female cheetah that they hope will breed with the male they already have...

Meanwhile, back to Python, I believe the name is Pandas is derived from the term "Panel data", but ultimately, it really **doesn't** matter where the name came from...!

Today we are going to see how to:

- input some data into a dataframe
- manipulate it in various ways
- output subsets of the data



Give feedback
Ask for help

---

## Getting started!

Like most Python modules or libraries, there's a common convention for importing:

```
# Start python3
```

```
python3
```

```
Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
```

```
# Import the modules we might need
```

```
import os, sys, subprocess
import numpy as np
import pandas as pd
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'pandas'
```

```
# If you find a module is missing, one option used to be to install a personal version!
```

```
subprocess.call("pip3 install pandas", shell = True)
```

```
error: externally-managed-environment

× This environment is externally managed
╰─> To install Python packages system-wide, try apt install
    python3-xyz, where xyz is the package you are trying to
    install.

    If you wish to install a non-Debian-packaged Python package,
    create a virtual environment using python3 -m venv path/to/venv.
    Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
    sure you have python3-full installed.

    If you wish to install a non-Debian packaged Python application,
    it may be easiest to use pipx install xyz, which will manage a
    virtual environment for you. Make sure you have pipx installed.

    See /usr/share/doc/python3.12/README.venv for more information.

note: If you believe this is a mistake, please contact your Python installation or OS dist
hint: See PEP 668 for the detailed specification.
```

Give feedback
Ask for help

```
exit()
```

```
# Try using apt to install it?
```

```
apt install pandas
```

```
E: Could not open lock file /var/lib/dpkg/lock-frontend - open (13: Permission denied)
E: Unable to acquire the dpkg frontend lock (/var/lib/dpkg/lock-frontend), are you root?
```

When `root` isnt around to help.....

Help link 1

Help link 2

```
# How about using pipx?
```

```
pipx
```

```
usage: pipx [-h] [--quiet] [--verbose] [--version]
            {install,uninject,inject,upgrade,upgrade-all,uninstall,uninstall-all,reinstall
            ...

Install and execute apps from Python packages.

Binaries can either be installed globally into isolated Virtual Environments
or run directly in a temporary Virtual Environment.

optional environment variables:
  PIPX_HOME               Overrides default pipx location. Virtual Environments will be inst
  PIPX_BIN_DIR            Overrides location of app installations. Apps are symlinked or cop
  PIPX_MAN_DIR            Overrides location of manual pages installations. Manual pages are
  PIPX_DEFAULT_PYTHON     Overrides default python used for commands.
  USE_EMOJI               Overrides emoji behavior. Default value varies based on platform.

options:
  -h, --help              show this help message and exit
  --quiet, -q             Give less output. May be used multiple times corresponding to the
  --verbose, -v           Give more output.
  --version               Print version and exit

subcommands:
  Get help for commands with pipx COMMAND --help

  {install,uninject,inject,upgrade,upgrade-all,uninstall,uninstall-all,reinstall,reinstall
    install               Install a package
    uninject              Uninstall injected packages from an existing Virtual Environment
    inject                Install packages into an existing Virtual Environment
    upgrade               Upgrade a package
    upgrade-all           Upgrade all packages. Runs `pip install -U <pkgname>` for each pac
    uninstall             Uninstall a package
    uninstall-all         Uninstall all packages
    reinstall             Reinstall a package
    reinstall-all         Reinstall all packages
    list                  List installed packages
    run                   Download the latest version of a package to a temporary virtual en
                          `__pypackages__` directory (experimental).
    runpip                Run pip in an existing pipx-managed Virtual Environment
    ensurepath             Ensure directories necessary for pipx operation are in your PATH
    environment           Print a list of environment variables and paths used by pipx.
    completions           Print instructions on enabling shell completions for pipx
```

Give feedback
Ask for help

```
# Check things are set up correctly?
```

```
pipx ensurepath
```

```
Success! Added /localdisk/home/aivens2/.local/bin to the PATH environment variable.

Consider adding shell completions for pipx. Run 'pipx completions' for instructions.

You will need to open a new terminal or re-login for the PATH changes to take effect.

Otherwise pipx is ready to go! ✨ 🌟 ✨
```

```
# Try installing something
```

```
pipx install ruff
```

```
  installed package ruff 0.14.4, installed using Python 3.12.3
  These apps are now globally available
    - ruff
⚠️  Note: '/localdisk/home/aivens2/.local/bin' is not on your PATH environment variable.
    These apps will not be globally accessible until your PATH is updated.
    Run `pipx ensurepath` to automatically add it, or manually modify your PATH in your sh
    config file (i.e. ~/.bashrc).
done! ✨ 🌟 ✨
```

```
# Exit the terminal and then come back in
```

```
exit
```

How else might we have done this, given that it is `~/.bashrc` that is being modified.....?!

```
# Quick check....
```

```
pipx ensurepath
```

```
/localdisk/home/aivens2/.local/bin is already in PATH.

⚠️  All pipx binary directories have been added to PATH. If you are sure you want to proce

Otherwise pipx is ready to go! ✨ 🌟 ✨
```

```
# Try our installed ruff programme!
```

```
ruff
```

```
Ruff: An extremely fast Python linter and code formatter.

Usage: ruff [OPTIONS] <COMMAND>

Commands:
  check    Run Ruff on the given files or directories
  rule     Explain a rule (or all rules)
  config   List or describe the available configuration options
  linter   List all supported upstream linters
  clean    Clear any caches in the current directory and any subdirectories
  format   Run the Ruff formatter on the given files or directories
  server   Run the language server
  analyze  Run analysis over Python source code
  version  Display Ruff's version
  help     Print this message or the help of the given subcommand(s)
```

Give feedback
Ask for help

```
Options:
  -h, --help     Print help
  -V, --version  Print version

Log levels:
  -v, --verbose  Enable verbose logging
  -q, --quiet    Print diagnostics, but nothing else
  -s, --silent   Disable all logging (but still exit with status code "1" upon detecting d

Global options:
      --config <CONFIG_OPTION>  Either a path to a TOML configuration file (`pyproject.tom
                                find in a `ruff.toml` configuration file) overriding a spe
                                option always take precedence over all configuration files
                                `--config`
      --isolated                Ignore all configuration files

For help with a specific command, see: `ruff help <command>`.
```

## # Let's try installing `pandas` now...

## pipx install pandas

```
Note: Dependent package 'numpy' contains 2 apps
  - f2py
  - numpy-config

No apps associated with package pandas. Try again with '--include-deps' to include apps of
dependent packages, which are listed above. If you are attempting to install a library, pi
should not be used. Consider using pip or a similar tool instead.
```

## # Ignored/missed their warning, tried with dependencies....

## pipx install --include-deps pandas

```
⚠️   File exists at /localdisk/home/aivens2/.local/bin/f2py and points to /localdisk/home/a
     /home/aivens2/.local/share/pipx/venvs/pandas/bin/f2py. Not modifying.
  installed package pandas 2.3.3, installed using Python 3.12.3
  These apps are now globally available
    - numpy-config
    - f2py (symlink missing or pointing to unexpected location)
done! ✨ 🌟 ✨
```

## # Are we all OK now?

## p3

```
Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
```

## import pandas

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'pandas'
```

## ...meh!... 😡

Give feedback
Ask for help

---

## Need/use a virtual environment?

If we need a special set of packages, and/or they cant be globally installed for some reason, then we can use what is called a *virtual environment*, a "bubble" set up the way we want it all to be...

```
# Let's be tidy!
mkdir ~/Exercises/Lecture17
cd ~/Exercises/Lecture17
```

```
# Create a virtual environment called playtime
python -m venv playtime
```

```
# What happened?!
ls -al playtime
```

```
total 4
drwxr-xr-x 5 aivens2 g_aivens2  98 Nov 10 09:27 .
drwxr-xr-x 3 aivens2 g_aivens2  30 Nov 10 09:27 ..
drwxr-xr-x 2 aivens2 g_aivens2 212 Nov 10 09:27 bin
drwxr-xr-x 3 aivens2 g_aivens2  32 Nov 10 09:27 include
drwxr-xr-x 3 aivens2 g_aivens2  32 Nov 10 09:27 lib
lrwxrwxrwx 1 aivens2 g_aivens2   3 Nov 10 09:27 lib64 -> lib
-rw-r--r-- 1 aivens2 g_aivens2 190 Nov 10 09:27 pyvenv.cfg
```

```
# What does the folder structure look like?
tree | head -n20
```

```
.
└── playtime
    ├── bin
    │   ├── activate
    │   ├── activate.csh
    │   ├── activate.fish
    │   ├── Activate.ps1
    │   ├── pip
```

Give feedback
Ask for help

```
│   ├── pip3
│   ├── pip3.12
│   ├── python -> /usr/bin/python
│   ├── python3 -> python
│   └── python3.12 -> python
├── include
│   └── python3.12
├── lib
│   └── python3.12
│       └── site-packages
│           ├── pip
│           │   ├── __init__.py
```

# How many files are there!?

ls -alR | wc -l

1774

# Does that not take up a whole pile of disk space?

du -hc --max-depth 1

```
15M     ./playtime
15M     .
15M     total
```

# Can we successfully install pandas now!?

playtime/bin/pip install pandas

```
Collecting pandas
  Using cached pandas-2.3.3-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.me
Collecting numpy>=1.26.0 (from pandas)
  Using cached numpy-2.3.4-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.met
Collecting python-dateutil>=2.8.2 (from pandas)
  Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting pytz>=2020.1 (from pandas)
  Using cached pytz-2025.2-py2.py3-none-any.whl.metadata (22 kB)
Collecting tzdata>=2022.7 (from pandas)
  Using cached tzdata-2025.2-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting six>=1.5 (from python-dateutil>=2.8.2->pandas)
  Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Using cached pandas-2.3.3-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (12.
Using cached numpy-2.3.4-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.6
Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Using cached pytz-2025.2-py2.py3-none-any.whl (509 kB)
Using cached tzdata-2025.2-py2.py3-none-any.whl (347 kB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: pytz, tzdata, six, numpy, python-dateutil, pandas
Successfully installed numpy-2.3.4 pandas-2.3.3 python-dateutil-2.9.0.post0 pytz-2025.2 s
```

# Try it now!

python

```
Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
```

Give feedback
Ask for help

```
import pandas
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'pandas'
```

D'oh!

```
# DOH! Need to activate our playtime environment!
source playtime/bin/activate
(playtime) aivens2@bioinfmsc8:~/Exercises/Lecture17$
```

```
# Try it now, this time we are in the environment!
python
```

```
Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
```

```
import pandas
```

```

```

```
pandas.__version__
```

```
'2.3.3'
```

Success!!

```
# Exiting our environment once we are all done
(playtime) aivens2@bioinfmsc8:~/Exercises/Lecture17$ deactivate
```

```
aivens2@bioinfmsc8:~/Exercises/Lecture17$
```

```
# Try it again, this time we are NOT in the environment!
python
```

```
Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
```

```
import pandas
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'pandas'
```

```
# Let's just get on with things....
source ~/Exercises/Lecture17/playtime/bin/activate
```

Give feedback
Ask for help

# python

```
Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
```

## # Import all the modules we want

```
import os, sys, subprocess
import numpy as np
import pandas as pd
```

Give feedback
Ask for help

---

## Series and dataframes

There are two types of things we need to know about: `Series` and `Dataframe` (name stolen from R). As with most things Python, there is lots of on-line documentation.

- https://pandas.pydata.org/pandas-docs/stable/reference/series.html
- https://pandas.pydata.org/pandas-docs/stable/reference/frame.html

A Series is like a list:

```
pd.Series(['a', 'b', 'c', 'd'])
```

```
0    a
1    b
2    c
3    d
dtype: object
```

However, unlike a list it has an explicit index (the thing on the left: 0, 1, 2, 3, essentially just the row numbers).
Also, the elements have a type (here it was `object` because they're strings).

If we put several Series objects together in a dict and assign them descriptive names then we can create a `DataFrame` object:

```
# Make a dataframe
s1 = pd.Series(['a', 'b', 'c', 'd'])
s2 = pd.Series([2,4,8,16])
pd.DataFrame( { 'letter' : s1, 'number' : s2 } )
```

```
        letter  number
0       a       2
1       b       4
2       c       8
3       d       16
```

Give feedback
Ask for help

We still have an explicit index (0, 1, 2, 3), AND the dataframe displays nicely as a table.

Give feedback
Ask for help

---

## Creating dataframes from files

For data manipulation/visualisation, we generally create a dataframe by reading data from a file.

There are many ways to input/output data... described in great detail on https://pandas.pydata.org/pandas-docs/stable/reference/io.html

Today we are going to use an NCBI file that contains information about the various eukaryotic genome sequences they have.

We can get the file using `wget`:

```
# I did this on 10th Nov 2025, it's not changed since last year 😭
```

```
subprocess.call('wget -qO eukaryotes.txt "ftp://ftp.ncbi.nlm.nih.gov/genomes/GENOME_REPORTS/eukaryotes.txt" ', shell=True)
```

```
0
```

```
os.system("wc -l eukaryotes.txt")
```

```
37951 eukaryotes.txt
0
```

```
os.system("head -n3 eukaryotes.txt")
```

```
#Organism/Name  TaxID   BioProject Accession    BioProject ID   Group   SubGroup        Size (Mb)       GC%     Assembly Accession      Replicons       WGS     Scaffol
Emiliania huxleyi CCMP1516      280463  PRJNA77753      77753   Protists        Other Protists  167.676 64.5    GCA_000372725.1 -       AHAL01  7795 38549     38554
Arabidopsis thaliana    3702    PRJNA10719      10719   Plants  Land Plants     119.669 36.0529 GCA_000001735.2 chromosome 1:NC_003070.9/CP002684.1; chromosome 2:NC_
0
```

First thing that we need to do is import the eukaryote genomes file into a dataframe table.

Notice that:

Give feedback
Ask for help

- the method we use is called `read_csv()` even though it's not actually a CSV file (it is actually tab-delimited, as we saw)
- we have to give the separator as a keyword argument
- pandas automatically treats the first line as a header row (column names)
- `df` is the conventional variable name for a dataframe
- we get an explicit index starting from zero

## # I need help with using read_csv

## help(pd.read_csv)

```
Help on function read_csv in module pandas.io.parsers:

read_csv(filepath_or_buffer, sep=',', dialect=None, compression='infer', doublequote=True,
    Read CSV (comma-separated) file into DataFrame

    Also supports optionally iterating or breaking of the file
    into chunks.

    Additional help can be found in the `online docs for IO Tools
    http://pandas.pydata.org/pandas-docs/stable/io.html`_.

    Parameters
    ----------
    filepath_or_buffer : string or file handle / StringIO
        The string could be a URL. Valid URL schemes include
        http, ftp, s3, and file. For file URLs, a
        host is expected. For instance, a local file could be
        file ://localhost/path/to/table.csv
    sep : string, default ','
        Delimiter to use. If sep is None, will try to automatically determine
        this. Regular expressions are accepted.
    engine : {'c', 'python'}
        Parser engine to use. The C engine is faster while the python engine is
        currently more feature-complete.
    lineterminator : string (length 1), default None
        Character to break file into lines. Only valid with C parser
    quotechar : string (length 1)
        The character used to denote the start and end of a quoted item. Quoted
        items can include the delimiter and it will be ignored.
    quoting : int or csv.QUOTE_* instance, default None
        Control field quoting behavior per ``csv.QUOTE_*`` constants. Use one of
        QUOTE_MINIMAL (0), QUOTE_ALL (1), QUOTE_NONNUMERIC (2) or QUOTE_NONE (3).
        Default (None) results in QUOTE_MINIMAL behavior.
    skipinitialspace : boolean, default False
        Skip spaces after delimiter
    escapechar : string (length 1), default None
        One-character string used to escape delimiter when quoting is QUOTE_NONE.
    dtype : Type name or dict of column -> type, default None

... LOTS more ...
```

## # We can also use DOT TAB TAB method to what is available

## pd.re # TAB TAB

```
pd.read_clipboard(  pd.read_feather(    pd.read_hdf(        pd.read_orc(        pd.read_sa
pd.read_csv(        pd.read_fwf(        pd.read_html(       pd.read_parquet(    pd.read_sp
pd.read_excel(      pd.read_gbq(        pd.read_json(       pd.read_pickle(     pd.read_sq
```

Give feedback
Ask for help

Give feedback
Ask for help

```
# Let's read the file in
df = pd.read_csv('eukaryotes.txt', sep="\t")
df
# Note the index on the left
```

```
                          #Organism/Name   TaxID  ...                                                  
0              Emiliania huxleyi CCMP1516  280463  ...                                                  
1                    Arabidopsis thaliana    3702  ...         The Arabidopsis Information Re           
2                    Neopyropia yezoensis    2788  ...                                      Oce         
3                             Glycine max    3847  ...             US DOE Joint Genome Instit           
4                     Medicago truncatula    3880  ...                                                  
...                                   ...     ...  ...                                                  
37945    Saccharomyces cerevisiae NC-02  947038  ...  Genome Sequencing Center (GSC) at Wa            
37946  Saccharomyces cerevisiae CLIB382  947035  ...  Genome Sequencing Center (GSC) at Wa            
37947           Saccharomyces cerevisiae    4932  ...                          Shanghai Jiao To        
37948           Saccharomyces cerevisiae    4932  ...                                   Chung-A        
37949           Saccharomyces cerevisiae    4932  ...                                      San         

[37950 rows x 19 columns]
```

```
# Similar things exist for "saving out" from a dataframe (once you have one! e.g. called df)
df.to_ # TAB TAB
```

```
df.to_clipboard(  df.to_feather(    df.to_json(       df.to_parquet(    df.to_sql(
df.to_csv(        df.to_gbq(        df.to_latex(      df.to_period(     df.to_stata(
df.to_dict(       df.to_hdf(        df.to_markdown(   df.to_pickle(     df.to_string(
df.to_excel(      df.to_html(       df.to_numpy(      df.to_records(    df.to_timestamp(
```

```
# How big is the data frame?
df.shape
```

```
(37950, 19)
```

```
# What are the column headings? Are the spaces an issue?
df.columns
```

```
Index(['#Organism/Name', 'TaxID', 'BioProject Accession', 'BioProject ID',
       'Group', 'SubGroup', 'Size (Mb)', 'GC%', 'Assembly Accession',
       'Replicons', 'WGS', 'Scaffolds', 'Genes', 'Proteins', 'Release Date',
       'Modify Date', 'Status', 'Center', 'BioSample Accession'],
      dtype='object')
```

Give feedback
Ask for help

## list(df.columns)

```
['#Organism/Name', 'TaxID', 'BioProject Accession', 'BioProject ID', 'Group', 'SubGroup',
```

## # Get the DOT TAB TAB help!

## df.

```
Display all 219 possibilities? (y or n)
df.Center                df.combine(              df.floordiv(             df.mean(                 df.replace(              df.to_dict(
df.Genes                 df.combine_first(        df.from_dict(            df.median(               df.resample(             df.to_excel(
df.Group                 df.compare(              df.from_records(         df.melt(                 df.reset_index(          df.to_feather(
df.Proteins              df.convert_dtypes(       df.ge(                   df.memory_usage(         df.rfloordiv(            df.to_gbq(
df.Replicons             df.copy(                 df.get(                  df.merge(                df.rmod(                 df.to_hdf(
df.Scaffolds             df.corr(                 df.groupby(              df.min(                  df.rmul(                 df.to_html(
df.Status                df.corrwith(             df.gt(                   df.mod(                  df.rolling(              df.to_json(
df.SubGroup              df.count(                df.head(                 df.mode(                 df.round(                df.to_latex(
df.T                     df.cov(                  df.hist(                 df.mul(                  df.rpow(                 df.to_markdown(
df.TaxID                 df.cummax(               df.iat                   df.multiply(             df.rsub(                 df.to_numpy(
df.WGS                   df.cummin(               df.idxmax(               df.ndim                  df.rtruediv(             df.to_orc(
df.abs()                 df.cumprod(              df.idxmin(               df.ne(                   df.sample(               df.to_parquet(
df.add(                  df.cumsum(               df.iloc                  df.nlargest(             df.select_dtypes(        df.to_period(
df.add_prefix(           df.describe(             df.index                 df.notna()               df.sem(                  df.to_pickle(
df.add_suffix(           df.diff(                 df.infer_objects(        df.notnull()             df.set_axis(             df.to_records(
df.agg(                  df.div(                  df.info(                 df.nsmallest(            df.set_flags(            df.to_sql(
df.aggregate(            df.divide(               df.insert(               df.nunique(              df.set_index(            df.to_stata(
df.align(                df.dot(                  df.interpolate(          df.pad(                  df.shape                 df.to_string(
df.all(                  df.drop(                 df.isetitem(             df.pct_change(           df.shift(                df.to_timestamp(
df.any(                  df.drop_duplicates(      df.isin(                 df.pipe(                 df.size                  df.to_xarray()
df.apply(                df.droplevel(            df.isna()                df.pivot(                df.skew(                 df.to_xml(
df.applymap(             df.dropna(               df.isnull()              df.pivot_table(          df.sort_index(           df.transform(
df.asfreq(               df.dtypes                df.items()               df.plot(                 df.sort_values(          df.transpose(
df.asof(                 df.duplicated(           df.iterrows()            df.pop(                  df.sparse                df.truediv(
df.assign(               df.empty                 df.itertuples(           df.pow(                  df.squeeze(              df.truncate(
df.astype(               df.eq(                   df.join(                 df.prod(                 df.stack(                df.tz_convert(
df.at                    df.equals(               df.keys()                df.product(              df.std(                  df.tz_localize(
df.at_time(              df.eval(                 df.kurt(                 df.quantile(             df.style                 df.unstack(
df.attrs                 df.ewm(                  df.kurtosis(             df.query(                df.sub(                  df.update(
df.axes                  df.expanding(            df.last(                 df.radd(                 df.subtract(             df.value_counts(
df.backfill(             df.explode(              df.last_valid_index()    df.rank(                 df.sum(                  df.values
df.between_time(         df.ffill(                df.le(                   df.rdiv(                 df.swapaxes(             df.var(
df.bfill(                df.fillna(               df.loc                   df.reindex(              df.swaplevel(            df.where(
df.bool()                df.filter(               df.lt(                   df.reindex_like(         df.tail(                 df.xs(
df.boxplot(              df.first(                df.map(                  df.rename(               df.take(
df.clip(                 df.first_valid_index()   df.mask(                 df.rename_axis(          df.to_clipboard(
df.columns               df.flags                 df.max(                  df.reorder_levels(       df.to_csv(
```

## # To examine bits of the dataframe we can use slices

## df[3:7]

```
                  #Organism/Name    TaxID  ...                                                  
3                     Glycine max     3847  ...            US DOE Joint Genome Institute (J
4              Medicago truncatula     3880  ...                                             
5             Solanum lycopersicum     4081  ...                          Solanaceae Genomics 
6  Hordeum vulgare subsp. vulgare   112509  ...  Leibniz Institute of Plant Genetics and Cr

[4 rows x 19 columns]
```

## # Or use built in `head()` and `tail()` methods

## df.head()

```
                #Organism/Name    TaxID  ...                                        Center  B
0  Emiliania huxleyi CCMP1516   280463  ...                                          JGI   
1         Arabidopsis thaliana     3702  ...  The Arabidopsis Information Resource (TAIR)
2         Neopyropia yezoensis     2788  ...                        Ocean University      
3                Glycine max      3847  ...    US DOE Joint Genome Institute (JGI-PGF)
4          Medicago truncatula     3880  ...                                        NCBI  

[5 rows x 19 columns]
```

Give feedback
Ask for help

```
# To get more information on the columns we have ended up with, use describe()
df.describe()
```

```
              TaxID  BioProject ID     Size (Mb)     Scaffolds
count  3.795000e+04   3.795000e+04  37950.000000  3.795000e+04
mean   5.613396e+05   6.506686e+05    581.685245  5.681285e+04
std    8.356700e+05   2.512643e+05   1429.903851  2.871059e+05
min    2.708000e+03   1.190000e+02      0.000520  1.000000e+00
25%    9.606000e+03   4.838862e+05     28.917325  1.460000e+02
50%    1.194430e+05   6.691910e+05     58.418100  9.940000e+02
75%    7.495890e+05   8.332210e+05    661.516750  8.963000e+03
max    3.135897e+06   1.153517e+06  91113.700000  1.167394e+07
```

By default, this just shows us the numerical columns.
Why no stats for GC content, number of genes, etc?
Because in the original file, unknown values are represented with a hyphen "-", and pandas doesn't know that that means "missing data".

We can tell it by using the `na_values` argument and giving a list of strings that mean "missing data".

```
# Re-read the file in, telling pandas about "missing data"
df = pd.read_csv('eukaryotes.txt', sep="\t", na_values=['-'])
df.describe()
```

```
              TaxID  BioProject ID     Size (Mb)           GC%     Scaffolds         Genes
count  3.795000e+04   3.795000e+04  37950.000000  37376.000000  3.795000e+04  7.431000e+03
mean   5.613396e+05   6.506686e+05    581.685245     42.458719  5.681285e+04  1.822345e+04
std    8.356700e+05   2.512643e+05   1429.903851      7.666657  2.871059e+05  8.392855e+04
min    2.708000e+03   1.190000e+02      0.000520      0.134504  1.000000e+00  1.000000e+00
25%    9.606000e+03   4.838862e+05     28.917325     37.400000  1.460000e+02  9.007500e+03
50%    1.194430e+05   6.691910e+05     58.418100     41.400000  9.940000e+02  1.242000e+04
75%    7.495890e+05   8.332210e+05    661.516750     48.200000  8.963000e+03  1.771950e+04
max    3.135897e+06   1.153517e+06  91113.700000     78.373100  1.167394e+07  4.736081e+06
```

Now we can see all sorts of interesting info for GC content, number of scaffolds, etc. Specifying missing data in this way is very useful - it means that pandas will automatically skip values when calculating things AND when plotting...

Give feedback
Ask for help

---

## Selecting from dataframes

What can we do with a dataframe once we have it!?

### We can extract individual columns

### df['Organism/Name']

```
Traceback (most recent call last):
  File "/localdisk/home/aivens2/Exercises/Lecture17/playtime/lib/python3.12/site-packages
    return self._engine.get_loc(casted_key)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "pandas/_libs/index.pyx", line 167, in pandas._libs.index.IndexEngine.get_loc
  File "pandas/_libs/index.pyx", line 196, in pandas._libs.index.IndexEngine.get_loc
  File "pandas/_libs/hashtable_class_helper.pxi", line 7088, in pandas._libs.hashtable.PyO
  File "pandas/_libs/hashtable_class_helper.pxi", line 7096, in pandas._libs.hashtable.PyO
KeyError: 'Organism/Name'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/localdisk/home/aivens2/Exercises/Lecture17/playtime/lib/python3.12/site-packages/
    indexer = self.columns.get_loc(key)
              ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/localdisk/home/aivens2/Exercises/Lecture17/playtime/lib/python3.12/site-packages/
    raise KeyError(key) from err
KeyError: 'Organism/Name'
```

### # Get the name right!

### df['#Organism/Name']

```
0               Emiliania huxleyi CCMP1516
1                     Arabidopsis thaliana
2                     Neopyropia yezoensis
3                              Glycine max
4                      Medicago truncatula
                         ...
37945       Saccharomyces cerevisiae NC-02
37946    Saccharomyces cerevisiae CLIB382
37947             Saccharomyces cerevisiae
37948             Saccharomyces cerevisiae
37949             Saccharomyces cerevisiae
Name: #Organism/Name, Length: 37950, dtype: object
```

Note the syntax: it looks like we're getting a single value from a dict called df, but

Give feedback
Ask for help

actually we're asking for that **column** from all rows. The result of doing this is a `Series` object.

Give feedback
Ask for help

We can count unique (non-redundant) values in a series and pandas will helpfully put the most frequent at the top.

```
df['#Organism/Name'].value_counts()
```

```
Saccharomyces cerevisiae             1412
Homo sapiens                         1215
Fusarium oxysporum                    398
Pyricularia oryzae                    375
Aspergillus fumigatus                 333
                                     ...
Tilapia sparmanii                       1
Lamprologus tigripictilis               1
Coptodon rendalli                       1
Heterochromis multidens                 1
Saccharomyces cerevisiae CLIB382        1
Name: #Organism/Name, Length: 17177, dtype: int64
```

```
# How many were there?
len(df['#Organism/Name'].value_counts())
```

```
17177
```

If we give a **list** of column names instead of a single one then we get back a `dataframe` object.

For subsequent examples we will generally use `head()` to make it easier to see the output, but all the rows are really there!

```
df[ ['#Organism/Name', 'Size (Mb)', 'GC%'] ].head()
```

| | #Organism/Name | Size (Mb) | GC% |
|---|---|---|---|
| 0 | Emiliania huxleyi CCMP1516 | 167.676 | 64.5000 |
| 1 | Arabidopsis thaliana | 119.669 | 36.0529 |
| 2 | Neopyropia yezoensis | 107.591 | 64.8454 |
| 3 | Glycine max | 978.942 | 35.1221 |
| 4 | Medicago truncatula | 430.008 | 33.4462 |

Give feedback
Ask for help

---

## Filtering dataframes

To figure out which rows satisfy some condition, we can use a special shorthand:

```
df['#Organism/Name'] == 'Arabidopsis thaliana'
```

```
0         False
1          True
2         False
3         False
4         False
          ...
37945     False
37946     False
37947     False
37948     False
37949     False
Name: #Organism/Name, Length: 37950, dtype: bool
```

```
# So how many of each did we get?
(df['#Organism/Name'] == 'Arabidopsis thaliana').value_counts()
```

```
False    37707
True       243
Name: #Organism/Name, dtype: int64
```

```
# Or use isin
df[df['#Organism/Name'].isin(["Arabidopsis thaliana"])].shape
(243, 19)
# Be specific with your query!
df[df['#Organism/Name'].isin(["Arabidopsis"])].shape
(0, 19)
```

This looks like we are comparing the whole column to the string, but pandas uses some special "Python magic" to make it compare each row and return a Series of True/False values. This isn't very useful on its own, but becomes useful when we slice the dataframe using those values, as shown below...

Give feedback
Ask for help

```
# Make a subset of the data
cress = df[df['#Organism/Name'] == 'Arabidopsis thaliana']
cress
```

```
             #Organism/Name  TaxID  ...                                                Center
1       Arabidopsis thaliana   3702  ...        The Arabidopsis Information Resource (TAIR)
20076   Arabidopsis thaliana   3702  ...         max planck institute for biology tuebingen
20080   Arabidopsis thaliana   3702  ...  Community-Consensus Arabidopsis Thaliana Refer...
23218   Arabidopsis thaliana   3702  ...          The Salk Institute for Biological Studies
23224   Arabidopsis thaliana   3702  ...         max planck institute for biology tuebingen
...                      ...    ...  ...                                                ...
34682   Arabidopsis thaliana   3702  ...                      MPI FOR DEVELOPMENTAL BIOLOGY
34683   Arabidopsis thaliana   3702  ...                      MPI FOR DEVELOPMENTAL BIOLOGY
34698   Arabidopsis thaliana   3702  ...                      MPI FOR DEVELOPMENTAL BIOLOGY
34699   Arabidopsis thaliana   3702  ...                      MPI FOR DEVELOPMENTAL BIOLOGY
34711   Arabidopsis thaliana   3702  ...                      MPI FOR DEVELOPMENTAL BIOLOGY

[243 rows x 19 columns]
```

Notice that the result is itself a dataframe, which we could save out as a csv file. See this for more details: https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.to_csv.html#pandas.DataFrame.to_csv

```
cress.to_csv("All_the_Arabidopsis_ones.tsv",sep="\t",header=True)
os.system("head -1 *; wc -l *")
```

```
==> All_the_Arabidopsis_ones.tsv <==
        #Organism/Name  TaxID   BioProject Accession    BioProject ID   Group   SubGroup

==> eukaryotes.txt <==
#Organism/Name  TaxID   BioProject Accession    BioProject ID   Group   SubGroup        Si
     244 All_the_Arabidopsis_ones.tsv
   37951 eukaryotes.txt
   38195 total
0
```

We can filter/extract to ask questions like:

```
# What are the sizes of all the Arabidopsis thaliana genomes?
df[df['#Organism/Name'] == 'Arabidopsis thaliana']['Size (Mb)']
```

```
1        119.669000
20076    143.501000
20080    142.481000
23218    132.081000
23224    139.933000
            ...
34682      2.035040
34683      0.844674
34698      0.838703
34699      0.325150
```

Give feedback
Ask for help

```
34711      0.847842
Name: Size (Mb), Length: 243, dtype: float64
```

```
# Store the sizes as a Series object, then describe() it
sizes = df[df['#Organism/Name'] == 'Arabidopsis thaliana']['Size (Mb)']
sizes
```

```
1        119.669000
20076    143.501000
20080    142.481000
23218    132.081000
23224    139.933000
            ...
34682      2.035040
34683      0.844674
34698      0.838703
34699      0.325150
34711      0.847842
Name: Size (Mb), Length: 243, dtype: float64
```

```
sizes.describe()
```

```
count    243.000000
mean      94.320730
std       60.632455
min        0.195283
25%        2.020030
50%      131.229000
75%      136.562000
max      244.583000
Name: Size (Mb), dtype: float64
```

Other comparison operators work too!

```
df[df['Size (Mb)'] > 10000]
```

```
              #Organism/Name  TaxID  ...                                                Cent
9           Triticum aestivum   4565  ...                                                 NC
587     Neoceratodus forsteri   7892  ...                             University of Konsta
609      Lepidosiren paradoxa   7883  ...  The Max Planck Institute of Molecular Cell Bio.
667           Bombina bombina   8345  ...                                                 NC
668    Ichthyophis bannanicus   8453  ...              Northwestern Polytechnical Universi
...                       ...    ...  ...                                                .
37511           Homo sapiens   9606  ...                                UNIVERSITY OF LEE
37515           Homo sapiens   9606  ...                                UNIVERSITY OF LEE
37520           Homo sapiens   9606  ...                                UNIVERSITY OF LEE
37523           Homo sapiens   9606  ...                                UNIVERSITY OF LEE
37528           Homo sapiens   9606  ...                                UNIVERSITY OF LEE

[112 rows x 19 columns]
```

```
# Use & to join multiple conditions
# note the brackets around the conditions
df[ (df['Size (Mb)'] > 10000) & (df['Status'] == 'Scaffold') ]
```

Give feedback
Ask for help

| | #Organism/Name | ... | BioSample Accession |
|---|---|---|---|
| 1182 | Pinus taeda | ... | SAMN02981512 |
| 1195 | Picea abies | ... | SAMEA3269885 |
| 1215 | Picea glauca | ... | SAMN02736787 |
| 1364 | Pseudotsuga menziesii | ... | SAMN03333061 |
| 1385 | Pinus lambertiana | ... | SAMN03354659 |
| 1530 | Sequoia sempervirens | ... | SAMN11654426 |
| 1603 | Picea sitchensis | ... | SAMN04296985 |
| 4429 | Larix sibirica | ... | SAMN07326255 |
| 5415 | Picea engelmannii | ... | SAMN10388286 |
| 6901 | Picea mariana | ... | SAMN14324163 |
| 11866 | Iris pallida | ... | SAMN28206045 |
| 14473 | Calotriton arnoldi | ... | SAMEA113426058 |
| 17354 | Sambucus nigra | ... | SAMEA7522178 |
| 18293 | Viscum album | ... | SAMEA7522516 |
| 19320 | Picea glauca | ... | SAMN01120252 |
| 20770 | Triticum dicoccoides | ... | SAMN13041385 |
| 22734 | Triticum dicoccoides | ... | SAMEA6062056 |
| 22872 | Allium cepa var. aggregatum | ... | SAMD00071601 |
| 25477 | Bubalus bubalis bubalis | ... | SAMEA8110765 |
| 28559 | Triticum aestivum | ... | SAMN41075130 |
| 28826 | Triticum aestivum | ... | SAMN41710599 |
| 29058 | Triticum aestivum | ... | SAMEA6062055 |
| 29065 | Triticum aestivum | ... | SAMEA104424263 |
| 29279 | Triticum aestivum | ... | SAMEA3663800 |
| 29287 | Triticum aestivum | ... | SAMEA6374024 |
| 29479 | Triticum aestivum | ... | SAMEA6374020 |
| 29490 | Triticum aestivum | ... | SAMEA6374022 |
| 29668 | Triticum aestivum | ... | SAMEA6374021 |
| 29678 | Triticum aestivum | ... | SAMEA6374023 |
| 30023 | Triticum aestivum | ... | SAMEA3269884 |
| 37447 | Homo sapiens | ... | SAMEA8949006 |
| 37452 | Homo sapiens | ... | SAMEA8948994 |
| 37455 | Homo sapiens | ... | SAMEA8949015 |
| 37463 | Homo sapiens | ... | SAMEA8949021 |
| 37464 | Homo sapiens | ... | SAMEA8949033 |
| 37468 | Homo sapiens | ... | SAMEA8949024 |
| 37471 | Homo sapiens | ... | SAMEA8949031 |
| 37475 | Homo sapiens | ... | SAMEA8949034 |
| 37480 | Homo sapiens | ... | SAMEA8949004 |
| 37488 | Homo sapiens | ... | SAMEA8948990 |
| 37511 | Homo sapiens | ... | SAMEA8949025 |
| 37515 | Homo sapiens | ... | SAMEA8949022 |
| 37520 | Homo sapiens | ... | SAMEA8948980 |
| 37523 | Homo sapiens | ... | SAMEA8948989 |
| 37528 | Homo sapiens | ... | SAMEA8949007 |

[45 rows x 19 columns]

Give feedback
Ask for help

---

## Your new best friend: the `apply()` method!

To run arbitrary expressions on rows, use the `apply()` method with a `lambda`.

What is a lambda?! Good description and examples here: https://www.afternerd.com/blog/python-lambdas/.
Easiest way to think of it is that it acts like a "mini-function" that we can use without having to write a bigger "proper" function.

`axis=1` means that we want to iterate over rows rather than columns.

The argument to the lambda expression will be a single row so we can treat it as a `dict` to get individual columns

```
# Make all the organism names upper case
df.apply(lambda x : x['#Organism/Name'].upper(), axis=1).head()
```

```
0    EMILIANIA HUXLEYI CCMP1516
1          ARABIDOPSIS THALIANA
2          NEOPYROPIA YEZOENSIS
3                   GLYCINE MAX
4           MEDICAGO TRUNCATULA
dtype: object
```

This comes in very handy for filtering with complex conditions!

```
# Find birds and mammals with "small" genome assemblies
# Which column do I need to look in?
df.columns
```

```
Index(['#Organism/Name', 'TaxID', 'BioProject Accession', 'BioProject ID',
       'Group', 'SubGroup', 'Size (Mb)', 'GC%', 'Assembly Accession',
       'Replicons', 'WGS', 'Scaffolds', 'Genes', 'Proteins', 'Release Date',
       'Modify Date', 'Status', 'Center', 'BioSample Accession'],
      dtype='object')
```

```
'Birds' in set(df['Group'])
```

Give feedback
Ask for help

False

'Birds' in set(df['SubGroup'])

True

'Mammals' in set(df['SubGroup'])

True

df[df.apply( lambda x : x['Size (Mb)'] < 500 and x['SubGroup'] in ['Birds'], axis=1 )]

| | #Organism/Name | TaxID | ... | Center | BioSample Accessio |
|---|---|---|---|---|---|
| 13438 | Pyrrhura rhodocephala | 311893 | ... | Iridian Genomes | SAMN2558240 |
| 15504 | Luscinia luscinia | 391696 | ... | Charles University | SAMN2609580 |
| 15505 | Luscinia megarhynchos | 383689 | ... | Charles University | SAMN2609580 |
| 17605 | Strigops habroptila | 2489341 | ... | Vertebrate Genomes Project | SAMN0994875 |
| 19254 | Ficedula albicollis | 59894 | ... | Uppsala University | SAMEA316639 |
| 20151 | Vidua chalybeata | 81927 | ... | Vertebrate Genomes Project | SAMN1225400 |
| 22467 | Calidris pugnax | 198806 | ... | Uppsala University | SAMN2668815 |

[7 rows x 19 columns]

df[df.apply( lambda x : x['Size (Mb)'] < 500 and x['SubGroup'] in ['Birds'], axis=1 )].shape[0]

7

df[df.apply( lambda x : x['Size (Mb)'] < 500 and x['SubGroup'] in ['Birds'], axis=1 )].shape[1]

19

df[df.apply( lambda x : x['Size (Mb)'] < 500 and x['SubGroup'] in ['Mammals'], axis=1 )].shape[0]

514

birds_and_mammals = df[df.apply( lambda x : x['Size (Mb)'] < 500 and x['SubGroup'] in ['Birds', 'Mammals'], axis=1 )]

birds_and_mammals

| | #Organism/Name | TaxID | ... | Center | BioSampl |
|---|---|---|---|---|---|
| 564 | Osphranter robustus | 9319 | ... | AUSTRALIAN NATIONAL UNIVERSITY | S |
| 578 | Isoodon macrourus macrourus | 120737 | ... | AUSTRALIAN NATIONAL UNIVERSITY | S |
| 670 | Phascogale tapoatafa | 9293 | ... | AUSTRALIAN NATIONAL UNIVERSITY | S |
| 671 | Potorous tridactylus | 9310 | ... | AUSTRALIAN NATIONAL UNIVERSITY | S |
| 672 | Pseudocheirus peregrinus | 9333 | ... | AUSTRALIAN NATIONAL UNIVERSITY | S |
| ... | ... | ... | ... | ... | |
| 37808 | Homo sapiens | 9606 | ... | STANFORD UNIVERSITY | S |

Give feedback
Ask for help

```
37811                  Homo sapiens    9606  ...           Harvard Medical School        S.
37812                  Homo sapiens    9606  ...       Baylor College of Medicine        S.
37814                  Homo sapiens    9606  ...              Pacific Biosciences        S.
37815                  Homo sapiens    9606  ...  Telomere-to-Telomere Consortium        S.

[521 rows x 19 columns]
```

## birds_and_mammals.shape

**(521, 19)**

## birds_and_mammals.shape[0]

**521**

Give feedback
Ask for help

## Sorting dataframes

To sort, there's a `sort_values()` method which mostly does what we want!

```
# Which are the largest genomes?
df.sort_values('Size (Mb)', ascending=False).head()
```

```
              #Organism/Name  TaxID  ...                                                Cente
18293           Viscum album   3972  ...                                WELLCOME SANGER INSTITUT
609     Lepidosiren paradoxa   7883  ...  The Max Planck Institute of Molecular Cell Bio..
19655   Lepidosiren paradoxa   7883  ...  The Max Planck Institute of Molecular Cell Bio..
1739   Protopterus annectens   7888  ...                                                NCB
587    Neoceratodus forsteri   7892  ...                            University of Konstan

[5 rows x 19 columns]
```

Notice that this returns a new dataframe (that we could have kept as a separate dataframe by assigning it to a new variable). Unlike lists, it doesn't affect the original (unless we pass `inplace=True`).

```
df.sort_values('Size (Mb)', ascending=False, inplace=True)
df
```

```
              #Organism/Name  TaxID  ...                                                Cente
18293           Viscum album   3972  ...                                WELLCOME SANGER INSTITUT
609     Lepidosiren paradoxa   7883  ...  The Max Planck Institute of Molecular Cell Bio..
19655   Lepidosiren paradoxa   7883  ...  The Max Planck Institute of Molecular Cell Bio..
1739   Protopterus annectens   7888  ...                                                NCB
587    Neoceratodus forsteri   7892  ...                            University of Konstan
...                      ...    ...  ...                                                ..
25442   Ornithodoros moubata   6938  ...                          Friedrich-Loeffler-Institu
24448   Ornithodoros moubata   6938  ...                          Friedrich-Loeffler-Institu
23448   Ornithodoros moubata   6938  ...                          Friedrich-Loeffler-Institu
24810   Ornithodoros moubata   6938  ...                          Friedrich-Loeffler-Institu
26358   Ornithodoros moubata   6938  ...                          Friedrich-Loeffler-Institu

[37950 rows x 19 columns]
```

```
df['Size (Mb)']
```

```
18293    91113.700000
609      87218.800000
19655    46159.900000
```

Give feedback
Ask for help

```
1739      40054.300000
587       34557.600000
              ...
25442         0.001600
24448         0.001196
23448         0.000548
24810         0.000545
26358         0.000520
Name: Size (Mb), Length: 37950, dtype: float64
```

```
# We can sort on multiple columns if we want
# sort by name, then within each name with highest GC first
df.sort_values(['#Organism/Name', 'GC%'], ascending=[True, False]).head()
```

```
                     #Organism/Name  ...  BioSample Accession
6194   Aaosphaeria arxii CBS 175.79  ...          SAMN05661102
11711      Aaosphaeria pasadenensis  ...          SAMN25226780
1795         Abelmoschus esculentus  ...          SAMN35805024
4315             Abeoforma whisleri  ...          SAMN06200030
8285             Abisara bifasciata  ...          SAMN12776983

[5 rows x 19 columns]
```

If we need to do something more complicated than this then we probably need to add a new column and then sort on it....

Give feedback
Ask for help

---

## Creating and updating columns

For simple cases, we can create a new column by assigning values to it using the syntax we've already seen:

```
# add a column for AT content
df['at_content'] = 1 - (df['GC%'] / 100)
```

```
# view the new column
df[ ['#Organism/Name', 'GC%', 'at_content'] ].head()
```

```
                #Organism/Name      GC%  at_content
18293            Viscum album  31.2000    0.688000
609     Lepidosiren paradoxa  40.4307    0.595693
19655   Lepidosiren paradoxa  40.3000    0.597000
1739   Protopterus annectens  40.4512    0.595488
587    Neoceratodus forsteri  44.9367    0.550633
```

This changes the dataframe in place.

```
# Having got used to referring to columns like this, this should work?
# make a genus column with the first part of the species name
df['genus'] = df['#Organism/Name'].split(' ')[0]
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/localdisk/home/aivens2/Exercises/Lecture17/playtime/lib/python3.12/site-packages/
    return object.__getattribute__(self, name)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'Series' object has no attribute 'split'. Did you mean: 'plot'?
```

but it doesn't - the "Python magic" only works on certain operators (mostly mathematical stuff).

`apply()` can help us out here.

```
df['genus'] = df.apply(lambda x : x['#Organism/Name'].split(' ')[0], axis=1)
```

Give feedback
Ask for help

## df['#Organism/Name', 'genus'] .head()

```
Traceback (most recent call last):
  File "/localdisk/home/aivens2/Exercises/Lecture17/playtime/lib/python3.12/site-packages/
    return self._engine.get_loc(casted_key)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "pandas/_libs/index.pyx", line 167, in pandas._libs.index.IndexEngine.get_loc
  File "pandas/_libs/index.pyx", line 196, in pandas._libs.index.IndexEngine.get_loc
  File "pandas/_libs/hashtable_class_helper.pxi", line 7088, in pandas._libs.hashtable.PyO
  File "pandas/_libs/hashtable_class_helper.pxi", line 7096, in pandas._libs.hashtable.PyO
KeyError: ('#Organism/Name', 'genus')

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/localdisk/home/aivens2/Exercises/Lecture17/playtime/lib/python3.12/site-packages/
    indexer = self.columns.get_loc(key)
              ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/localdisk/home/aivens2/Exercises/Lecture17/playtime/lib/python3.12/site-packages/
    raise KeyError(key) from err
KeyError: ('#Organism/Name', 'genus')
```

## # If you want to view columns, they need to be formatted as a list!

## df[ ['#Organism/Name', 'genus'] ].head()

| | #Organism/Name | genus |
|---|---|---|
| 18293 | Viscum album | Viscum |
| 609 | Lepidosiren paradoxa | Lepidosiren |
| 19655 | Lepidosiren paradoxa | Lepidosiren |
| 1739 | Protopterus annectens | Protopterus |
| 587 | Neoceratodus forsteri | Neoceratodus |

Give feedback
Ask for help

## Descriptive statistics

We can take various measurements of a Series

```
print(df[ ['Size (Mb)', 'GC%'] ].mean())
```

```
Size (Mb)    581.685245
GC%           42.458719
dtype: float64
```

```
print(df[ ['Size (Mb)', 'GC%'] ].median())
```

```
Size (Mb)    58.4181
GC%          41.4000
dtype: float64
```

```
print(df[ ['Size (Mb)', 'GC%'] ].std())
```

```
Size (Mb)    1429.903851
GC%             7.666657
dtype: float64
```

```
# Pearson correlation between columns too!
df[ ['Size (Mb)', 'Genes', 'Proteins', 'GC%'] ].corr()
```

```
            Size (Mb)     Genes  Proteins       GC%
Size (Mb)    1.000000  0.121138  0.471012 -0.134400
Genes        0.121138  1.000000  0.206491 -0.052944
Proteins     0.471012  0.206491  1.000000 -0.209682
GC%         -0.134400 -0.052944 -0.209682  1.000000
```

Give feedback
Ask for help

## Indexing

So far we have not really talked about indexing, but it turns out to be quite important. The index is the thing on the left of the ouptut: currently just the row number assigned when we first read the dataframe in.

```
df.head()
```

```
              #Organism/Name  TaxID BioProject Accession  ...  BioSample Accession at_cont
18293           Viscum album   3972            PRJEB64803  ...          SAMEA7522516   0.688
609     Lepidosiren paradoxa   7883           PRJNA808322  ...          SAMN26083907   0.595
19655   Lepidosiren paradoxa   7883           PRJNA808321  ...          SAMN26083907   0.597
1739   Protopterus annectens   7888           PRJNA754246  ...          SAMN17890435   0.595
587    Neoceratodus forsteri   7892           PRJNA644903  ...          SAMN15471169   0.550

[5 rows x 21 columns]
```

If there's a unique identifier somewhere in the data, then we can use that as the index by passing `index_col` when we read the data in.

```
# Let's read PART OF IT in again, but specify what we want as the index
df_mini = pd.read_csv('eukaryotes.txt', sep="\t", na_values=['-'], index_col='Assembly Accession').head()
df_mini
```

```
                              #Organism/Name  ...  BioSample Accession
Assembly Accession                            ...
GCA_000372725.1     Emiliania huxleyi CCMP1516  ...         SAMN02744062
GCA_000001735.2           Arabidopsis thaliana  ...         SAMN03081427
GCA_009829735.1           Neopyropia yezoensis  ...         SAMN13316713
GCA_000004515.5                     Glycine max  ...         SAMN00002965
GCA_003473485.2            Medicago truncatula  ...         SAMN08400029

[5 rows x 18 columns]
```

```
# We could also use a non-unique index without any issues
df['#Organism/Name'].value_counts().head()
```

Give feedback
Ask for help

```
Homo sapiens                1060
Saccharomyces cerevisiae    1412
Homo sapiens                1215
Fusarium oxysporum           398
Pyricularia oryzae           375
Aspergillus fumigatus        333
Name: #Organism/Name, dtype: int64
```

```
df_org = pd.read_csv('eukaryotes.txt', sep="\t", na_values=['-'], index_col='#Organism/Name')
```

```
df_org.head(2)
```

```
                             TaxID  ... BioSample Accession
#Organism/Name                      ...
Emiliania huxleyi CCMP1516  280463  ...        SAMN02744062
Arabidopsis thaliana          3702  ...        SAMN03081427

[2 rows x 18 columns]
```

```
# Everything from the original file is still there...
```

```
df_org.shape
```

```
(37950, 18)
```

```
# Different one this time, indexing on sequencing Center
```

```
df_center = pd.read_csv('eukaryotes.txt', sep="\t", na_values=['-'], index_col='Center')
```

```
df_center.head(16)
```

```
                                                                #Organism/Name  ...  B
Center                                                                          ...
JGI                                                        Emiliania huxleyi CCMP1516  ...
The Arabidopsis Information Resource (TAIR)                       Arabidopsis thaliana  ...
Ocean University                                                  Neopyropia yezoensis  ...
US DOE Joint Genome Institute (JGI-PGF)                                  Glycine max  ...
NCBI                                                            Medicago truncatula  ...
Solanaceae Genomics Project                                     Solanum lycopersicum  ...
Leibniz Institute of Plant Genetics and Crop Pl...  Hordeum vulgare subsp. vulgare  ...
NCBI                                               Oryza sativa Japonica Group  ...
MaizeGDB                                                                 Zea mays  ...
NCBI                                                           Triticum aestivum  ...
Broad Institute                                              Pneumocystis carinii B80  ...
S. pombe European Sequencing Consortium (EUPOM)            Schizosaccharomyces pombe  ...
Saccharomyces Genome Database                   Saccharomyces cerevisiae S288C  ...
NCBI                                              Aspergillus nidulans FGSC A4  ...
J. Craig Venter Institute                         Aspergillus fumigatus Af293  ...
Broad Institute                                       Neurospora crassa OR74A  ...

[16 rows x 18 columns]
```

```
# Is the column still there in the dataframe?
```

```
df_center[ ['Center'] ]
```

Give feedback
Ask for help

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/localdisk/home/s0000000/.local/lib/python3.8/site-packages/pandas/core/frame.py",
    indexer = self.loc._get_listlike_indexer(key, axis=1)[1]
  File "/localdisk/home/s0000000/.local/lib/python3.8/site-packages/pandas/core/indexing.p
    self._validate_read_indexer(keyarr, indexer, axis)
  File "/localdisk/home/s0000000/.local/lib/python3.8/site-packages/pandas/core/indexing.p
    raise KeyError(f"None of [{key}] are in the [{axis name}]")
KeyError: "None of [Index(['Center'], dtype='object')] are in the [columns]"
```

That was why there were 18 columns, instead of 19!

**df_center.columns**

```
Index(['#Organism/Name', 'TaxID', 'BioProject Accession', 'BioProject ID',
       'Group', 'SubGroup', 'Size (Mb)', 'GC%', 'Assembly Accession',
       'Replicons', 'WGS', 'Scaffolds', 'Genes', 'Proteins', 'Release Date',
       'Modify Date', 'Status', 'BioSample Accession'],
      dtype='object')
```

Rather than reading in the data again, we could set the index to be one of the existing columns...

**df_rejigged = df_center.set_index('BioSample Accession')**

**df_rejigged**

| BioSample Accession | #Organism/Name | TaxID | ... | Modify Date | Status |
|---|---|---|---|---|---|
| SAMN02744062 | Emiliania huxleyi CCMP1516 | 280463 | ... | 2014/08/01 | Scaffold |
| SAMN03081427 | Arabidopsis thaliana | 3702 | ... | 2022/10/20 | Chromosome |
| SAMN13316713 | Neopyropia yezoensis | 2788 | ... | 2020/01/06 | Chromosome |
| SAMN00002965 | Glycine max | 3847 | ... | 2021/03/10 | Chromosome |
| SAMN08400029 | Medicago truncatula | 3880 | ... | 2018/09/12 | Chromosome |
| ... | ... | ... | ... | ... | ... |
| SAMN14365262 | Saccharomyces cerevisiae NC-02 | 947038 | ... | 2020/06/23 | Scaffold |
| SAMN00198998 | Saccharomyces cerevisiae CLIB382 | 947035 | ... | 2014/08/04 | Scaffold |
| SAMN20308677 | Saccharomyces cerevisiae | 4932 | ... | 2023/03/27 | Chromosome |
| SAMN26740367 | Saccharomyces cerevisiae | 4932 | ... | 2024/05/24 | Chromosome |
| SAMEA3138293 | Saccharomyces cerevisiae | 4932 | ... | 2009/07/02 | Chromosome |

[37950 rows x 17 columns]

... however ...

**df_rejigged.columns**

```
Index(['#Organism/Name', 'TaxID', 'BioProject Accession', 'BioProject ID',
       'Group', 'SubGroup', 'Size (Mb)', 'GC%', 'Assembly Accession',
       'Replicons', 'WGS', 'Scaffolds', 'Genes', 'Proteins', 'Release Date',
       'Modify Date', 'Status'],
      dtype='object')
```

**len(df_center.columns)**

```
18
```

**len(df_rejigged.columns)**

Give feedback
Ask for help

```
17
```

... but luckily ...

**df_center.index**

```
Index(['JGI', 'The Arabidopsis Information Resource (TAIR)',
       'Ocean University', 'US DOE Joint Genome Institute (JGI-PGF)', 'NCBI',
       'Solanaceae Genomics Project',
       'Leibniz Institute of Plant Genetics and Crop Plant Research', 'NCBI',
       'MaizeGDB', 'NCBI',
       ...
       'University of Adelaide',
       'Genome Sequencing Center (GSC) at Washington University (WashU) School of Medicine
       'Genome Sequencing Center (GSC) at Washington University (WashU) School of Medicine
       'Genome Sequencing Center (GSC) at Washington University (WashU) School of Medicine
       'Genome Sequencing Center (GSC) at Washington University (WashU) School of Medicine
       'Genome Sequencing Center (GSC) at Washington University (WashU) School of Medicine
       'Genome Sequencing Center (GSC) at Washington University (WashU) School of Medicine
       'Shanghai Jiao Tong University', 'Chung-Ang University',
       'Sanger Institute'],
      dtype='object', name='Center', length=37950)
```

We can also make our own index out of items in the data. This we do by assigning to the special variable `df.index` as we have just seen above, which lets us do things like....

**# Start again (just to be safe!): re-read data in with row number as default index**

**df = pd.read_csv('eukaryotes.txt', sep="\t", na_values=['-'])**

**# We want to create a new index which is made up of the Organism name and the accession number**

**# Check our command out first**

**df.apply(lambda x : "{} ({})".format(x['#Organism/Name'], x['BioSample Accession']), axis=1)**

```
0              Emiliania huxleyi CCMP1516 (SAMN02744062)
1                    Arabidopsis thaliana (SAMN03081427)
2                    Neopyropia yezoensis (SAMN13316713)
3                             Glycine max (SAMN00002965)
4                     Medicago truncatula (SAMN08400029)
                              ...
37945      Saccharomyces cerevisiae NC-02 (SAMN14365262)
37946    Saccharomyces cerevisiae CLIB382 (SAMN00198998)
37947           Saccharomyces cerevisiae (SAMN20308677)
37948           Saccharomyces cerevisiae (SAMN26740367)
37949           Saccharomyces cerevisiae (SAMEA3138293)
Length: 37950, dtype: object
```

Give feedback
Ask for help

```
# OK, so let's make the index, as that seems to work
```

```
df.index = df.apply(lambda x : "{} ({})".format(x['#Organism/Name'], x['BioSample Accession']), axis=1)
```

```
df
```

```
                                                                #Organism/Name  ...  Bi
Emiliania huxleyi CCMP1516 (SAMN02744062)              Emiliania huxleyi CCMP1516  ...
Arabidopsis thaliana (SAMN03081427)                          Arabidopsis thaliana  ...
Neopyropia yezoensis (SAMN13316713)                          Neopyropia yezoensis  ...
Glycine max (SAMN00002965)                                             Glycine max  ...
Medicago truncatula (SAMN08400029)                            Medicago truncatula  ...
...                                                                              ...  ...
Saccharomyces cerevisiae NC-02 (SAMN14365262)      Saccharomyces cerevisiae NC-02  ...
Saccharomyces cerevisiae CLIB382 (SAMN00198998)  Saccharomyces cerevisiae CLIB382  ...
Saccharomyces cerevisiae (SAMN20308677)                  Saccharomyces cerevisiae  ...
Saccharomyces cerevisiae (SAMN26740367)                  Saccharomyces cerevisiae  ...
Saccharomyces cerevisiae (SAMEA3138293)                  Saccharomyces cerevisiae  ...

[37950 rows x 19 columns]
```

```
df.shape
```

```
(37950, 19)
```

```
df.index.value_counts()
```

```
Solanum verrucosum (SAMEA104077226)        19
Homo sapiens (SAMN03283347)                13
Taeniopygia guttata (SAMN09946140)         11
Mustela putorius (SAMEA5818864)            10
Homo sapiens (SAMN03255769)                 9
                                           ..
Arthroderma sp. MBC 290 (SAMN30838493)      1
Lipomyces yarrowii (SAMN20341228)           1
[Candida] tunisiensis (SAMN20341277)        1
Beauveria australis (SAMN30838441)          1
Saccharomyces cerevisiae (SAMEA3138293)     1
Length: 35010, dtype: int64
```

```
df['TaxID'].head(3)
```

```
Emiliania huxleyi CCMP1516 (SAMN02744062)    280463
Arabidopsis thaliana (SAMN03081427)            3702
Neopyropia yezoensis (SAMN13316713)            2788
Name: TaxID, dtype: int64
```

```
df[0:3]['TaxID']
```

```
Emiliania huxleyi CCMP1516 (SAMN02744062)    280463
Arabidopsis thaliana (SAMN03081427)            3702
Neopyropia yezoensis (SAMN13316713)            2788
Name: TaxID, dtype: int64
```

```
df['TaxID'].values[0:3]
```

```
array([280463, 3702, 2788])
```

```
# Is it like a dictionary, got keys too!? No...
```

Give feedback
Ask for help

```
df['TaxID'].keys[0:3]
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'method' object is not subscriptable
```

Give feedback
Ask for help

## Selections using `iloc`

### # Single rows : first row

### df.iloc[0]

```
#Organism/Name          Emiliania huxleyi CCMP1516
TaxID                                       280463
BioProject Accession                     PRJNA77753
BioProject ID                                77753
Group                                     Protists
SubGroup                           Other Protists
Size (Mb)                                 167.676
GC%                                          64.5
Assembly Accession                  GCA_000372725.1
Replicons                                       NaN
WGS                                          AHAL01
Scaffolds                                    7795
Genes                                     38549.0
Proteins                                  38554.0
Release Date                           2013/04/19
Modify Date                            2014/08/01
Status                                   Scaffold
Center                                        JGI
BioSample Accession                    SAMN02744062
Name: Emiliania huxleyi CCMP1516 (SAMN02744062), dtype: object
```

### # Single rows : last row

### df.iloc[-1]

```
#Organism/Name                                  Saccharomyces cerevisiae
TaxID                                                               4932
BioProject Accession                                          PRJNA13838
BioProject ID                                                      13838
Group                                                             Fungi
SubGroup                                                      Ascomycetes
Size (Mb)                                                        0.586761
GC%                                                              38.5921
Assembly Accession                                      GCA_000091065.2
Replicons               chromosome III:X59720.2; chromosome VI:D50617.1
WGS                                                                 NaN
Scaffolds                                                             2
Genes                                                             155.0
Proteins                                                          298.0
Release Date                                                   1992/03/16
Modify Date                                                    2009/07/02
Status                                                         Chromosome
Center                                                    Sanger Institute
BioSample Accession                                          SAMEA3138293
Name: Saccharomyces cerevisiae (SAMEA3138293), dtype: object
```

Give feedback
Ask for help

## # Single columns : first column

## df.iloc[:,0]

```
Emiliania huxleyi CCMP1516 (SAMN02744062)                 Emiliania huxleyi CCMP1516
Arabidopsis thaliana (SAMN03081427)                             Arabidopsis thaliana
Neopyropia yezoensis (SAMN13316713)                             Neopyropia yezoensis
Glycine max (SAMN00002965)                                                Glycine max
Medicago truncatula (SAMN08400029)                               Medicago truncatula
                                                                             ...
Saccharomyces cerevisiae NC-02 (SAMN14365262)          Saccharomyces cerevisiae NC-02
Saccharomyces cerevisiae CLIB382 (SAMN00198998)     Saccharomyces cerevisiae CLIB382
Saccharomyces cerevisiae (SAMN20308677)                    Saccharomyces cerevisiae
Saccharomyces cerevisiae (SAMN26740367)                    Saccharomyces cerevisiae
Saccharomyces cerevisiae (SAMEA3138293)                    Saccharomyces cerevisiae
Name: #Organism/Name, Length: 37950, dtype: object
```

## # Single columns : last column

## df.iloc[:,-1]

```
Emiliania huxleyi CCMP1516 (SAMN02744062)          SAMN02744062
Arabidopsis thaliana (SAMN03081427)                SAMN03081427
Neopyropia yezoensis (SAMN13316713)                SAMN13316713
Glycine max (SAMN00002965)                         SAMN00002965
Medicago truncatula (SAMN08400029)                 SAMN08400029
                                                       ...
Saccharomyces cerevisiae NC-02 (SAMN14365262)      SAMN14365262
Saccharomyces cerevisiae CLIB382 (SAMN00198998)    SAMN00198998
Saccharomyces cerevisiae (SAMN20308677)            SAMN20308677
Saccharomyces cerevisiae (SAMN26740367)            SAMN26740367
Saccharomyces cerevisiae (SAMEA3138293)            SAMEA3138293
Name: BioSample Accession, Length: 37950, dtype: object
```

## # Multiple row and column selection examples

```
df.iloc[0:5]                 # first five rows of dataframe
df.iloc[:, 0:2]              # first two columns of data frame with all rows
df.iloc[ [0,3,6,24], [0,5,6] ] # 1st, 4th, 7th, 25th row + 1st 6th 7th columns.
df.iloc[0:5, 5:8]            # first 5 rows and 5th 6th 7th columns
```

## # The actual outputs

## df.columns

```
Index(['#Organism/Name', 'TaxID', 'BioProject Accession', 'BioProject ID',
       'Group', 'SubGroup', 'Size (Mb)', 'GC%', 'Assembly Accession',
       'Replicons', 'WGS', 'Scaffolds', 'Genes', 'Proteins', 'Release Date',
       'Modify Date', 'Status', 'Center', 'BioSample Accession'],
      dtype='object')
```

## df.columns[5:8]

## Index(['SubGroup', 'Size (Mb)', 'GC%'], dtype='object')

## df.iloc[0:5, 5:8]

```
                                                  SubGroup  Size (Mb)       GC%
Emiliania huxleyi CCMP1516 (SAMN02744062)  Other Protists    167.676  64.5000
Arabidopsis thaliana (SAMN03081427)           Land Plants    119.669  36.0529
Neopyropia yezoensis (SAMN13316713)                 Other    107.591  64.8454
```

Give feedback
Ask for help

| | | | |
|---|---|---|---|
| Glycine max (SAMN00002965) | Land Plants | 978.942 | 35.1221 |
| Medicago truncatula (SAMN08400029) | Land Plants | 430.008 | 33.4462 |

Give feedback
Ask for help

---

The Pandas `loc` indexer can be used with DataFrames for two different use cases:

- Selecting rows by label/index (as we have just done using `iloc`)
- Selecting rows with a boolean / conditional lookup (i.e. True of False)

The loc indexer is used with the same syntax as iloc:

```
data.loc[<row selection>, <column selection>]
```

```
# Let's try it
```

```
df.loc['Glycine max (SAMN00002965)']
```

```
#Organism/Name                                                Glycine max
TaxID                                                            3847
BioProject Accession                                        PRJNA19861
BioProject ID                                                    19861
Group                                                          Plants
SubGroup                                                   Land Plants
Size (Mb)                                                      978.942
GC%                                                            35.1221
Assembly Accession                                      GCA_000004515.5
Replicons              chromosome 1:NC_016088.4/CM000834.4; chromosom...
WGS                                                             ACUP04
Scaffolds                                                          347
Genes                                                          59046.0
Proteins                                                       74248.0
Release Date                                                2010/01/05
Modify Date                                                 2021/03/10
Status                                                      Chromosome
Center                              US DOE Joint Genome Institute (JGI-PGF)
BioSample Accession                                       SAMN00002965
Name: Glycine max (SAMN00002965), dtype: object
```

```
# Let's try it with more than one
```

```
df.loc[ ['Glycine max (SAMN00002965)','Solanum lycopersicum (SAMN02981290)'] ]
```

```
                                        #Organism/Name  ...  BioSample Accession
Glycine max (SAMN00002965)                  Glycine max  ...         SAMN00002965
Solanum lycopersicum (SAMN02981290)  Solanum lycopersicum  ...         SAMN02981290

[2 rows x 19 columns]
```

Give feedback
Ask for help

```
# Quick reminder, what columns do we have to choose from?
```

```
df.columns
```

```
Index(['#Organism/Name', 'TaxID', 'BioProject Accession', 'BioProject ID',
       'Group', 'SubGroup', 'Size (Mb)', 'GC%', 'Assembly Accession',
       'Replicons', 'WGS', 'Scaffolds', 'Genes', 'Proteins', 'Release Date',
       'Modify Date', 'Status', 'Center', 'BioSample Accession'],
      dtype='object')
```

```
# We choose some columns
```

```
df.loc[ ['Glycine max (SAMN00002965)','Solanum lycopersicum (SAMN02981290)'],['TaxID','Size (Mb)','GC%','Genes'] ]
```

```
                                    TaxID  Size (Mb)      GC%    Genes
Glycine max (SAMN00002965)           3847    978.942  35.1221  59046.0
Solanum lycopersicum (SAMN02981290)  4081    827.963  35.6925  31219.0
```

```
# Other way of doing it, same result!
```

```
df.loc[ ['Glycine max (SAMN00002965)','Solanum lycopersicum (SAMN02981290)'],list(df.columns[ [1,6,7,12] ])]
```

```
                                    TaxID  Size (Mb)      GC%    Genes
Glycine max (SAMN00002965)           3847    978.942  35.1221  59046.0
Solanum lycopersicum (SAMN02981290)  4081    827.963  35.6925  31219.0
```

```
# Other way of doing it, different column order!
```

```
df.loc[ ['Glycine max (SAMN00002965)','Solanum lycopersicum (SAMN02981290)'],list(df.columns[ [6,7,1,12] ])]
```

```
                                  Size (Mb)      GC%  TaxID    Genes
Glycine max (SAMN00002965)          978.942  35.1221   3847  59046.0
Solanum lycopersicum (SAMN02981290)  827.963  35.6925   4081  31219.0
```

```
# We choose a particular organism
```

```
df.loc[df['#Organism/Name']=='Venturia inaequalis']
```

```
                                        #Organism/Name  ...  BioSample Accession
Venturia inaequalis (SAMN07816553)  Venturia inaequalis  ...          SAMN07816553
Venturia inaequalis (SAMN07816628)  Venturia inaequalis  ...          SAMN07816628
Venturia inaequalis (SAMN07816619)  Venturia inaequalis  ...          SAMN07816619
Venturia inaequalis (SAMN03081268)  Venturia inaequalis  ...          SAMN03081268
Venturia inaequalis (SAMN03080669)  Venturia inaequalis  ...          SAMN03080669
...                                                    ...  ...                   ...
Venturia inaequalis (SAMN07816556)  Venturia inaequalis  ...          SAMN07816556
Venturia inaequalis (SAMN07816552)  Venturia inaequalis  ...          SAMN07816552
Venturia inaequalis (SAMN07816568)  Venturia inaequalis  ...          SAMN07816568
Venturia inaequalis (SAMN07816549)  Venturia inaequalis  ...          SAMN07816549
Venturia inaequalis (SAMN07816547)  Venturia inaequalis  ...          SAMN07816547

[88 rows x 19 columns]
```

```
# Columns chosen, first three rows
```

```
df.loc[df['#Organism/Name']=='Venturia inaequalis',['TaxID','Size (Mb)','GC%','Genes'] ][0:3]
```

Give feedback
Ask for help

```
                                    TaxID  Size (Mb)   GC%  Genes
Venturia inaequalis (SAMN07816553)   5025    72.7916  43.1    NaN
Venturia inaequalis (SAMN07816628)   5025    51.7000   NaN    NaN
Venturia inaequalis (SAMN07816619)   5025    60.8229   NaN    NaN
```

## # Columns chosen, first three rows, different way

```
df.loc[df['#Organism/Name']=='Venturia inaequalis',['TaxID','Size (Mb)','GC%','Genes'] ].head(3)
```

```
                                    TaxID  Size (Mb)   GC%  Genes
Venturia inaequalis (SAMN07816553)   5025    72.7916  43.1    NaN
Venturia inaequalis (SAMN07816628)   5025    51.7000   NaN    NaN
Venturia inaequalis (SAMN07816619)   5025    60.8229   NaN    NaN
```

## # Looking for cows (*Bos taurus*)

```
df.loc[df['#Organism/Name']=='Bos taurus']
```

```
                            #Organism/Name  TaxID  ...
Bos taurus (nan)                Bos taurus   9913  ...
Bos taurus (SAMN43074752)       Bos taurus   9913  ...                                  Univers
Bos taurus (SAMN43074753)       Bos taurus   9913  ...                                  Univers
Bos taurus (SAMEA115082612)     Bos taurus   9913  ...                NORWEGIAN UNIVERSITY O
Bos taurus (SAMEA14214903)      Bos taurus   9913  ...                                     IN
Bos taurus (SAMN32907642)       Bos taurus   9913  ...                           Seoul Nati
Bos taurus (SAMN16377989)       Bos taurus   9913  ...
Bos taurus (SAMN26241768)       Bos taurus   9913  ...                           Northwest
Bos taurus (SAMN37529654)       Bos taurus   9913  ...                        Yunnan Agricult
Bos taurus (SAMEA7047893)       Bos taurus   9913  ...                                THE R
Bos taurus (SAMEA7051353)       Bos taurus   9913  ...                                THE R
Bos taurus (SAMN16425342)       Bos taurus   9913  ...
Bos taurus (SAMN35152401)       Bos taurus   9913  ...  Indian Institute of Science Educat
Bos taurus (SAMEA9551350)       Bos taurus   9913  ...                                the e
Bos taurus (SAMN02898106)       Bos taurus   9913  ...  Cattle Genome Sequencing Internati
Bos taurus (SAMN02898106)       Bos taurus   9913  ...  Center for Bioinformatics and Comp
Bos taurus (SAMN35642812)       Bos taurus   9913  ...

[17 rows x 19 columns]
```

## # Looking for a Polish cows

```
df.loc[df['#Organism/Name']=='Polish cow']
```

```
Empty DataFrame
Columns: [#Organism/Name, TaxID, BioProject Accession, BioProject ID, Group, SubGroup, Siz
Index: []
```

None found thankfully!

Give feedback
Ask for help

## Some other quick cool things

### df['Genes'].value_counts()

```
37.0       49
13.0       43
6907.0      5
6912.0      5
6911.0      5
           ..
15911.0     1
15454.0     1
15762.0     1
15393.0     1
155.0       1
Name: Genes, Length: 6150, dtype: int64
```

### df[['#Organism/Name','Genes']].value_counts()

```
Aspergillus fumigatus      9481.0     5
Ophidiomyces ophidiicola   6892.0     5
Saccharomyces cerevisiae   6376.0     4
Aspergillus fumigatus      9570.0     4
                           9626.0     4
                                     ..
Eudyptes sclateri          13760.0    1
Eudyptes schlegeli         16825.0    1
Eudyptes robustus          16687.0    1
Eudyptes pachyrhynchus     18731.0    1
fungal sp. No.11243        9730.0     1
Length: 7170, dtype: int64
```

### Aggregate

### df['Genes'].agg('min')

**1.0**

### df['Genes'].agg('max')

**4736081.0**

### df['Genes'].agg(['max','min','mean','std'])

```
max     4.736081e+06
min     1.000000e+00
mean    1.822345e+04
std     8.392855e+04
Name: Genes, dtype: float64
```

### type(df['Genes'].agg(['max','min','mean','std']))

Give feedback
Ask for help

<class 'pandas.core.series.Series'>

## groupby

## df.groupby('#Organism/Name').mean()

| #Organism/Name | TaxID | BioProject ID | Size (Mb) | GC% | Scaffolds | G |
|---|---|---|---|---|---|---|
| Aaosphaeria arxii CBS 175.79 | 1450172.0 | 625762.0 | 38.901000 | 49.90 | 55.0 | 142 |
| Aaosphaeria pasadenensis | 3025424.0 | 800051.0 | 38.797400 | 49.70 | 120.0 | |
| Abelmoschus esculentus | 455045.0 | 985739.0 | 1194.600000 | 33.80 | 78.0 | |
| Abeoforma whisleri | 749232.0 | 360047.0 | 101.554000 | 31.40 | 71510.0 | |
| Abisara bifasciata | 2614024.0 | 606954.0 | 332.742000 | 32.30 | 754990.0 | |
| ... | ... | ... | ... | ... | ... | |
| uncultured Thalassiosira | 654899.0 | 955529.0 | 30.739600 | 51.60 | 5894.0 | |
| uncultured Trebouxiophyceae | 167235.0 | 809297.0 | 4.756060 | 73.00 | 2846.0 | |
| uncultured Tremellales | 227714.0 | 809291.0 | 14.909500 | 56.00 | 1783.0 | |
| uncultured ciliate | 187299.0 | 941656.0 | 17.869020 | 31.40 | 1146.2 | |
| uncultured eukaryote | 100272.0 | 850925.8 | 14.964724 | 59.32 | 908.6 | |

[17177 rows x 7 columns]

## type(df.groupby('#Organism/Name').mean())

<class 'pandas.core.frame.DataFrame'>

## df.groupby('#Organism/Name').mean().iloc[:,0:3]

| #Organism/Name | TaxID | BioProject ID | Size (Mb) |
|---|---|---|---|
| Aaosphaeria arxii CBS 175.79 | 1450172.0 | 625762.0 | 38.901000 |
| Aaosphaeria pasadenensis | 3025424.0 | 800051.0 | 38.797400 |
| Abelmoschus esculentus | 455045.0 | 985739.0 | 1194.600000 |
| Abeoforma whisleri | 749232.0 | 360047.0 | 101.554000 |
| Abisara bifasciata | 2614024.0 | 606954.0 | 332.742000 |
| ... | ... | ... | ... |
| uncultured Thalassiosira | 654899.0 | 955529.0 | 30.739600 |
| uncultured Trebouxiophyceae | 167235.0 | 809297.0 | 4.756060 |
| uncultured Tremellales | 227714.0 | 809291.0 | 14.909500 |
| uncultured ciliate | 187299.0 | 941656.0 | 17.869020 |
| uncultured eukaryote | 100272.0 | 850925.8 | 14.964724 |

[17177 rows x 3 columns]

## df.groupby('#Organism/Name').mean().iloc[:,0:3].head(10)

| #Organism/Name | TaxID | BioProject ID | Size (Mb) |
|---|---|---|---|
| Aaosphaeria arxii CBS 175.79 | 1450172.0 | 625762.0 | 38.90100 |
| Aaosphaeria pasadenensis | 3025424.0 | 800051.0 | 38.79740 |
| Abelmoschus esculentus | 455045.0 | 985739.0 | 1194.60000 |
| Abeoforma whisleri | 749232.0 | 360047.0 | 101.55400 |
| Abisara bifasciata | 2614024.0 | 606954.0 | 332.74200 |
| Abortiporus biennis | 137743.0 | 444028.0 | 33.11860 |
| Abramis brama | 38527.0 | 946171.0 | 1018.32975 |
| Abrostola tripartita | 938171.0 | 715706.5 | 379.47800 |
| Abrostola triplasia | 254365.0 | 870861.5 | 361.63200 |
| Abrus precatorius | 3816.0 | 510631.0 | 347.23000 |

## df.groupby('#Organism/Name').mean().iloc[0:14:,0:3]

| #Organism/Name | TaxID | BioProject ID | Size (Mb) |
|---|---|---|---|
| Aaosphaeria arxii CBS 175.79 | 1450172.0 | 625762.0 | 38.90100 |
| Aaosphaeria pasadenensis | 3025424.0 | 800051.0 | 38.79740 |

Give feedback
Ask for help

| | | | |
|---|---|---|---|
| Abelmoschus esculentus | 455045.0 | 985739.0 | 1194.60000 |
| Abeoforma whisleri | 749232.0 | 360047.0 | 101.55400 |
| Abisara bifasciata | 2614024.0 | 606954.0 | 332.74200 |
| Abortiporus biennis | 137743.0 | 444028.0 | 33.11860 |
| Abramis brama | 38527.0 | 946171.0 | 1018.32975 |
| Abrostola tripartita | 938171.0 | 715706.5 | 379.47800 |
| Abrostola triplasia | 254365.0 | 870861.5 | 361.63200 |
| Abrus precatorius | 3816.0 | 510631.0 | 347.23000 |
| Abrus pulchellus subsp. cantoniensis | 858909.0 | 812623.5 | 381.26800 |
| Abscondita cerata | 2740424.0 | 699421.0 | 967.16200 |
| Abscondita terminalis | 2069292.0 | 556938.0 | 499.65300 |
| Absidia glauca | 4829.0 | 317991.0 | 48.74640 |

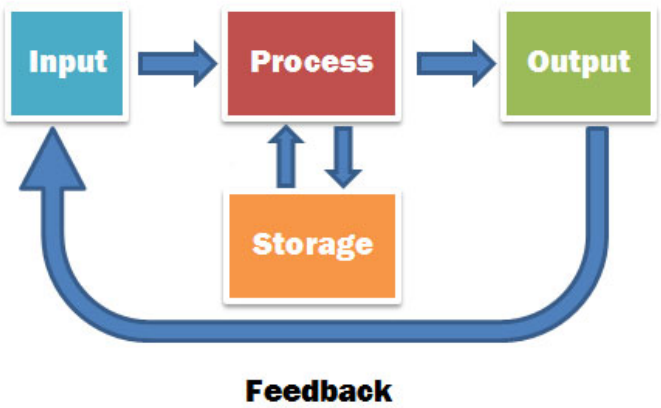
---

## Exercises: your turn!

## Let's try and be tidy...

Make a directory within your homespace for today's work (if you didnt already!), and change directory to it.

```
mkdir ~/Exercises/Lecture17
cd ~/Exercises/Lecture17
```




## Sending yet another email from the server

Today we are going to send yet another email from the bioinformatics server! You are going to use it to send me (and also to yourself) a short message about

Give feedback
Ask for help

your recent git usage.

Here is what you need to do, BEFORE you start any of today's exercises!

1. log in to the server
2. change directory to wherever you have your **Lecture Exercises git repository**
3. copy paste the following 3 lines of commands into your terminal window

```
DATE=$(date)
git log | grep "Date:" | egrep "Oct|Nov" > message_to_al_3
mail -s "${USER} ${HOSTNAME} ${DATE}" < message_to_al_3 aivens2@ed.ac.uk,${USER}@ed.ac.uk
```

**Mail sent, now check your email!**

Download the latest `eukaryotes.txt` equivalent file from NCBI (or alternatively, use the one provided in `/localdisk/data/BPSM/Lecture17`)

Use a pandas dataframe to address the following questions:

- how many fungal species have genomes bigger than 100Mb? What are their names?
- how many of each Kingdom/group (plants, animals, fungi and protists) have been sequenced?
- which *Heliconius* species genomes have been sequenced?
- which sequencing centre has sequenced the most plant genomes? the most insect genomes?
- add a column giving the number of proteins per gene. Which genomes have at least 10% more proteins than genes?

---

Give feedback
Ask for help

## Possible answers

Give feedback
Ask for help

```
# First import the libraries
```

```
import os, sys, re, numpy as np
import pandas as pd
```

```
# Get the file
```

```
os.mkdir("${HOME}/Exercises/Lecture17")
```

```
os.chdir("${HOME}/Exercises/Lecture17")
```

```
# If you opted for the new version, you could call it eukaryotes.tsv as its tab-separated values?
```

```
os.system("wget -qO eukaryotes.tsv 'ftp://ftp.ncbi.nlm.nih.gov/genomes/GENOME_REPORTS/eukaryotes.txt'")
```

```
# Read the file into a dataframe
```

```
df = pd.read_csv('eukaryotes.tsv', sep="\t", na_values=['-'])
```

```
# Create a new index which is made up of the species name and the accession number
```

```
df.index=df.apply(lambda x : "{} ({})".format(x['#Organism/Name'], x['BioSample Accession']), axis=1)
```

```
# What are the column headers
```

```
df.columns
```

```
Index(['#Organism/Name', 'TaxID', 'BioProject Accession', 'BioProject ID',
       'Group', 'SubGroup', 'Size (Mb)', 'GC%', 'Assembly Accession',
       'Replicons', 'WGS', 'Scaffolds', 'Genes', 'Proteins', 'Release Date',
       'Modify Date', 'Status', 'Center', 'BioSample Accession'],
      dtype='object')
```

```
# How many fungal species have genomes bigger than 100Mb?
# use two conditions joined with &
len( df[ (df['Group'] == 'Fungi') & (df['Size (Mb)'] > 100) ] )
```

```
218
```

```
# or we could use apply
len(df[df.apply(lambda x : x['Group'] == 'Fungi' and x['Size (Mb)'] > 100, axis=1)])
```

```
218
```

```
# What are their names?
```

Give feedback
Ask for help

```
# Start with the same query as before, then get the names column
big_fungi = df[(df['Group'] == 'Fungi') & (df['Size (Mb)'] > 100)]
```

```
# Transform the series object into a simple list of strings
list(big_fungi['#Organism/Name'])
```

```
['Blumeria graminis f. sp. triticale', 'Puccinia triticina', 'Tuber melanosporum Mel28'
...
'Rhizophagus irregularis DAOM 197198w', 'Rhizophagus irregularis DAOM 197198w', 'Rhizophagus irregularis']
```

```
# Nicer when it is sorted!
sorted(list(big_fungi['#Organism/Name']))
```

```
['Acaulospora colombiana', 'Acaulospora morrowiae', 'Albatrellus ellisii',
...
'Verticillium longisporum', 'Wilcoxina mikolae CBS 423.85', 'Zopfia rhizophila CBS 207.26']
```

```
# How many genomes of each major group (plants, animals, fungi and protists) have been sequenced, and how many unique organisms?
# We can figure out the answer for a single group
len(df[df['Group'] == 'Plants'])
```

```
4390
```

```
len(df[df['Group'] == 'plants'])
```

```
0
```

```
len(df[df['Group'].str.lower() == 'plants'])
```

```
4390
```

```
len(df[df['Group'].str.lower() == 'PLANTS'])
```

```
0
```

```
len(df[df['Group'].str.upper() == 'PLANTS'])
```

```
4390
```

```
# Remember that we are in Python3, so we can use a normal loop
for Group in ['Plants', 'Animals', 'Fungi', 'Protists']:
    count = len(df[df['Group'] == Group])
# To count unique ones, we could get the names and turn them into a set
    count_unique = len(set(df[df['Group'] == Group]['#Organism/Name']))
# OR we could use the drop_duplicates method which may be clearer to read
    count_unique = len(df[df['Group'] == Group].drop_duplicates('#Organism/Name'))
    print("{} genomes for {} ({} unique)".format(count, Group, count_unique))
```

Give feedback
Ask for help

```
4390 genomes for Plants (1912 unique)
14495 genomes for Animals (8190 unique)
16876 genomes for Fungi (6060 unique)
1987 genomes for Protists (871 unique)
```

## # Which *Heliconius* species genomes have been sequenced, and how many scaffolds is each assembly in?

## # here we HAVE to use apply

```
hel = df[df.apply(lambda x : x['#Organism/Name'].startswith('Heliconius'), axis=1)]
hel[ ['#Organism/Name', 'Scaffolds'] ]
```

| | #Organism/Name | Scaffolds |
|---|---|---|
| Heliconius cydno galanthus (SAMN25599327) | Heliconius cydno galanthus | 1341 |
| Heliconius melpomene melpomene (SAMEA2272046) | Heliconius melpomene melpomene | 4309 |
| Heliconius numata (SAMN16746774) | Heliconius numata | 15243 |
| Heliconius pachinus (SAMEA3670564) | Heliconius pachinus | 283902 |
| Heliconius ismenius (SAMEA3670537) | Heliconius ismenius | 160084 |
| ... | ... | ... |
| Heliconius melpomene amaryllis (SAMEA1094479) | Heliconius melpomene amaryllis | 104829 |
| Heliconius melpomene amaryllis (SAMEA1094482) | Heliconius melpomene amaryllis | 46387 |
| Heliconius melpomene amaryllis (SAMEA1094488) | Heliconius melpomene amaryllis | 116151 |
| Heliconius melpomene aglaope (SAMEA1094472) | Heliconius melpomene aglaope | 49473 |
| Heliconius melpomene aglaope (SAMEA1094478) | Heliconius melpomene aglaope | 105664 |

[182 rows x 2 columns]

## # Which center has sequenced the most plant genomes?

## df[df['Group'] == 'Plants']['Center'].value_counts().head()

```
USDA-ARS                                                    314
Leibniz Institute of Plant Genetics and Crop Plant Research 185
WELLCOME SANGER INSTITUTE                                   167
Iridian Genomes                                             151
NCBI                                                        141
Name: Center, dtype: int64
```

## # Which center has sequenced the most insect genomes?

## (df['Group'] == 'Insects').value_counts()

```
False    26070
Name: Group, dtype: int64
```

## # Oops!

## (df['SubGroup'] == 'Insects').value_counts()

```
False    32858
True      5092
Name: SubGroup, dtype: int64
```

## df[df['SubGroup'] == 'Insects']['Center'].value_counts().head()

Give feedback
Ask for help

```
WELLCOME SANGER INSTITUTE            1637
Florida Museum of Natural History     816
Stanford University                   274
NCBI                                  249
University of Cambridge, UK           141
Name: Center, dtype: int64
```

```
# Add a column giving the number of proteins per gene
```

```
# first, we do the calculation
```

```
df['Proteins'] / df['Genes']
```

```
Emiliania huxleyi CCMP1516 (SAMN02744062)          1.000130
Arabidopsis thaliana (SAMN03081427)                1.259821
Neopyropia yezoensis (SAMN13316713)                     NaN
Glycine max (SAMN00002965)                         1.257460
Medicago truncatula (SAMN08400029)                 1.078153
                                                       ...
Saccharomyces cerevisiae NC-02 (SAMN14365262)           NaN
Saccharomyces cerevisiae CLIB382 (SAMN00198998)         NaN
Saccharomyces cerevisiae (SAMN20308677)                 NaN
Saccharomyces cerevisiae (SAMN26740367)                 NaN
Saccharomyces cerevisiae (SAMEA3138293)            1.922581
Length: 37950, dtype: float64
```

```
# We can assign this to a new column
```

```
df['Proteins per gene'] = df['Proteins'] / df['Genes']
```

```
# Show the results
```

```
df[ ['#Organism/Name', 'Group', 'Proteins per gene'] ].head()
```

```
                                                    #Organism/Name      Group  Proteins
Emiliania huxleyi CCMP1516 (SAMN02744062)  Emiliania huxleyi CCMP1516  Protists
Arabidopsis thaliana (SAMN03081427)              Arabidopsis thaliana    Plants
Neopyropia yezoensis (SAMN13316713)              Neopyropia yezoensis     Other
Glycine max (SAMN00002965)                                Glycine max    Plants
Medicago truncatula (SAMN08400029)                 Medicago truncatula    Plants
```

```
# Which genomes have at least 10% more proteins than genes?
```

```
df[df['Proteins per gene'] >= 1.1][ ['#Organism/Name',
'Genes','Proteins','Proteins per gene'] ].head()
```

```
                                          #Organism/Name    Genes  Proteins  Proteins per
Arabidopsis thaliana (SAMN03081427)  Arabidopsis thaliana  38311.0   48265.0           1.2
Glycine max (SAMN00002965)                   Glycine max  59046.0   74248.0           1.2
Solanum lycopersicum (SAMN02981290)  Solanum lycopersicum  31219.0   37604.0           1.2
Zea mays (SAMEA5569141)                         Zea mays  49897.0   57578.0           1.1
Aedes aegypti (SAMN07177802)                 Aedes aegypti  19623.0   28317.0           1.4
```

```
df[df['Proteins per gene'] >= 1.1][ ['#Organism/Name',
'Genes','Proteins','Proteins per gene'] ]
```

```
                                                                                #Or
Arabidopsis thaliana (SAMN03081427)                                          Arabidops
Glycine max (SAMN00002965)
Solanum lycopersicum (SAMN02981290)                                          Solanum l
Zea mays (SAMEA5569141)
```

Give feedback
Ask for help

```
Aedes aegypti (SAMN07177802)                                                    Ae
...
Fusarium oxysporum f. sp. vasinfectum 25433 (SA...        Fusarium oxysporum f. sp. vasinf
Fusarium oxysporum f. sp. raphani 54005 (SAMN02...            Fusarium oxysporum f. sp. ra
Fusarium oxysporum f. sp. conglutinans race 2 5...  Fusarium oxysporum f. sp. conglutinans
Fusarium oxysporum NRRL 32931 (SAMN02981331)                          Fusarium oxysporum
Saccharomyces cerevisiae (SAMEA3138293)                                    Saccharomyces

[1070 rows x 4 columns]
```

## len(df[ df['Proteins per gene'] >= 1.1])

**1070**

## # More than 2.5 proteins per gene...

## df[df['Proteins per gene'] >= 2.5][ ['#Organism/Name', 'Genes','Proteins','Proteins per gene'] ]

| | #Organism/Name | Genes |
|---|---|---|
| Gallus gallus (SAMN15960293) | Gallus gallus | 25638.0 |
| Pan troglodytes (SAMN30216104) | Pan troglodytes | 41815.0 |
| Citrus sinensis (SAMN19611724) | Citrus sinensis | 55825.0 |
| Triticum turgidum subsp. durum (SAMEA104312462) | Triticum turgidum subsp. durum | 63993.0 |
| Enteromyxum leei (SAMN03701405) | Enteromyxum leei | 5.0 |
| Linum tenue (SAMEA110422887) | Linum tenue | 52052.0 |
| Phyllostomus discolor (SAMN14734468) | Phyllostomus discolor | 21058.0 |
| Citrus sinensis (SAMN19611724) | Citrus sinensis | 55880.0 |
| Schistosoma japonicum (SAMN19770425) | Schistosoma japonicum | 9768.0 |
| Engystomops pustulosus (SAMN13066191) | Engystomops pustulosus | 18426.0 |
| Ascaphus truei (SAMN12123227) | Ascaphus truei | 17155.0 |
| Gallus gallus (SAMN15960293) | Gallus gallus | 25612.0 |
| Zea mays (SAMN04296295) | Zea mays | 39320.0 |

---

# git and GitHub time!

## # Local repository actions (the absolute minimum set of commands required here...)

## git st

```
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    ../Lecture04/CodeFiles.tar.gz
        modified:   ../Lecture06/run_this_script_v1

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ../BN.out
        ../BN.sh
        ../Lecture04/Als_ICA/
        ../Lecture04/all_the_things_I_did
        ../Lecture04/outs/
        ../Lecture05/Country.Mozambique.details
        ../Lecture07/Cosmoscarta.nuc.fa
        ../Lecture07/Cosmoscarta.nuc.gis
```

Give feedback
Ask for help

```
        ../Lecture07/myfileofaccessions.txt
        ../Lecture12/AJ223353.fasta
        ../Lecture12/AJ223353.genbank
        ../Lecture12/AJ223353_noheader.fasta
        ../Lecture12/AJ223353_noheader.fasta2
        ../Lecture12/AJ223353_noheader.fasta3
        ../Lecture12/All_exons.fasta
        ../Lecture12/All_noncodings.fasta
        ../Lecture12/local_exon01.fasta
        ../Lecture12/local_exon02.fasta
        ../Lecture12/local_intron01.fasta
        ../Lecture12/my_dna.txt
        ../Lecture12/plain_genomic_seq.txt
        ../Lecture12/quick_blast_check.out
        ../Lecture12/remote_exon01.fasta
        ../Lecture12/remote_noncoding01.fasta
        ../Lecture12/remote_noncoding02.fasta
        ../Lecture12/shutilversion.txt
        ../Lecture13/100_199/
        ../Lecture13/200_299/
        ../Lecture13/300_399/
        ../Lecture13/400_499/
        ../Lecture13/500_599/
        ../Lecture13/600_699/
        ../Lecture13/700_799/
        ../Lecture13/800_899/
        ../Lecture13/900_999/
        ../Lecture13/Cleaned_sequences.txt
        ../Lecture13/Cleaned_sequences2.txt
        ../Lecture13/Cleaned_sequences3.txt
        ../Lecture13/Exercise2_coding_seqence.fasta
        ../Lecture13/Lecture13.tar.gz
        ../Lecture13/dna_files/
        ../Lecture13/exons.txt
        ../Lecture13/genomic_dna2.txt
        ../Lecture13/input.txt
        ../Lecture13/remote_exon01.fasta
        ../Lecture14/__pycache__/
        ../Lecture14/data.csv
        ../Lecture16/__pycache__/
        ./

no changes added to commit (use "git add" and/or "git commit -a")
```

## ls -1

```
All_the_Arabidopsis_ones.tsv
eukaryotes.txt
myPandacommands.py
playtime
```

## # You might want to add other files?

## git add *.py

## warning: in the working copy of 'Lecture17/myPandacommands.py', CRLF will be replaced by LF the next time Git touches it

## git commit -m "Script from the python pandas lecture"

```
[master 097800e] Script from the python pandas lecture
 1 file changed, 96 insertions(+)
 create mode 100644 Lecture17/myPandacommands.py
```

## # Push our local repository to GitHub

## git remote

Give feedback
Ask for help

```
AllLectures
Lecture07
origin
```

## git push AllLectures master

```
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 256 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 1.67 KiB | 857.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/B123456-2121/BPSM_LectureStuff.git
   1d2319e..097800e  master -> master
```

## git lo

```
097800e (HEAD -> master, AllLectures/master) Script from the python pandas lecture
1d2319e Scripts from the python dictionaries lecture
c49df45 Scripts from the python functions lecture
3b8bf41 My scripts from the python conditionals exercises Py04
8f58f4c Scripts from the python lists and loops lecture 3
e444ef8 Scripts from the second BPSM python lecture
6d6f5e7 My python scripts from the first BPSM python lecture
eae65c6 (Lecture07/master) edirect seqs related stuff added
eea4f5e (Lec_6_branch) Kept bash script output files from Lecture 06
fbe5cc0 Added run_this_script_v1 from BPSM Lecture06
ce2c15a nematode BLASTDB stuff added
19cdd2d Added the example_people_data.tsv file from Lecture03
137c3db Decided to keep blastoutput2.out
08d379b Decided to keep myinputfile.tsv
c9b5b18 First attempt at parsing BLAST data
bbf825c My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeea I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```

## Have your say...

The "Postbox of Truth": is your chance to give anonymous "in course" feedback/comments. Your comments are completely anonymous and confidential, so say what you think, irrespective of whether the comments are positive or otherwise! You can use this multiple times too: I use it throughout the course to enable you to give context-relevant feedback/make suggestions.

Just click the image!

## Sources used